

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

LOS Assignment_Theory_6

Reg. Number	Name	Branch
261100690017	**Shravya B**	**Cybersecurity**

03/11/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

ANALYSIS OF SSH AUTHENTICATION EVENTS USING BASH, POWER FILTERS, SED AND AWK

LOS theory Assignment_6

1. Introduction and Selected Cybersecurity Scenario

This assignment analyses Secure Shell (SSH) authentication events from a small, synthetic Linux authentication log. SSH is commonly used by administrators and users to access Linux systems remotely. Authentication logs record successful and unsuccessful login attempts. Repeated failures, attempts involving invalid usernames, and unexpected successful logins may indicate password guessing, account enumeration or unauthorized access.

The security significance is that an attacker who guesses a valid password may gain access to a system, steal information, install malware or move to other systems. Analysing authentication logs helps identify repeated attempts and provides evidence for further investigation.

2. How the Log File Was Obtained

A safe, authorized simulation was used instead of collecting logs from a public or institutional system. The file sample_auth.log was manually prepared with representative SSH authentication messages in the general format used by Linux authentication logs (such as /var/log/auth.log).

The simulated environment represents a single Linux host named kali. The records include failed password attempts, invalid usernames, successful password authentication, successful public-key authentication and one pre-authentication connection closure. The source IP addresses belong to documentation-only address ranges (RFC 5737): 192.0.2.0/24, 198.51.100.0/24 and 203.0.113.0/24. They are examples, not real attacker addresses.

Procedure: create sample_auth.log in the assignment folder, insert the synthetic records, and run the supplied Bash script against that file. No live SSH service was attacked, no external host was scanned and no real credentials were collected. Since this is a simulation, there is no downloaded dataset URL or dataset access date.

3. Structure of the Log File and Important Fields

Each record is a single SSH-related syslog message. A typical record is:

> **Nov 03 09:00:01 kali sshd[2101]: Failed password for invalid user admin from 203.0.113.10 port 51001 ssh2**

Important fields:

- Nov 03 – month and day of the event.
- 09:00:01 – time at which the event was recorded.
- kali – hostname of the Linux system that generated the message.
- sshd[2101] – SSH daemon process name and process ID.
- Failed password / Accepted publickey – authentication result or event description.
- admin – attempted username; in an invalid-user message it follows the words 'invalid user'.
- 203.0.113.10 – source IP address from which the connection originated.
- port 51001 – source-side network port used for that connection.

- ssh2 – SSH protocol version label.

The supplied file contains 20 records: 15 failed authentication events, 4 successful authentication events and 1 other SSH connection event. The analysis script reports the same authentication totals.

4. Cybersecurity Investigation Questions

- Which source IP addresses generated repeated failed SSH authentication attempts?
- Which source IP addresses meet the chosen suspicious-activity threshold?
- Were any successful SSH logins recorded, and which usernames and source addresses were involved?
- Did the log contain attempts using invalid or commonly targeted account names?
- Do the records show patterns that could be consistent with password guessing or account enumeration?

5. Tools and Analysis Method

The analysis combines Bash scripting with command-line filters. The main script calls sed, grep, awk, sort, uniq, cut and cat. The separate AWK file performs structured extraction and counting. This demonstrates a pipeline in which each tool performs a specific task.

The workflow is: normalize whitespace with sed; select SSH authentication messages with grep; save the selected records; parse event details with awk; count failures by source IP using a pipeline; and classify sources that reach the threshold.

```
sed -E 's/[[:space:]]+/ /g' sample_auth.log
```

The expression replaces one or more whitespace characters with a single space. This makes spacing consistent for later field-based processing. The original log is not modified.

```
grep -E 'sshd.*(Failed password|Invalid user|Accepted password|Accepted publickey)' normalized.log
```

grep selects lines that contain SSH daemon authentication outcomes. The -E option enables extended regular expressions.

```
grep ... | awk '{for (i=1; i<=NF; i++) if ($i=="from") print $(i+1)}' | sort | uniq -c | sort -nr
```

This pipeline extracts source IP addresses from failed events, sorts them, counts repeated values with uniq -c, and sorts the counts in descending numerical order.

```
awk '$1 >= 5 {print $2 " : " $1 " failed attempts"}' failures_by_ip.txt
```

This AWK condition selects source IPs with five or more failed attempts.

6. Bash Script and Separate AWK Script

The submission includes two executable analysis files: analyse_ssh.sh and analyse_ssh.awk. The Bash script controls the workflow and output files. The AWK script extracts timestamps, usernames, source IPs and authentication outcomes, then totals failed and successful events.

Main Bash script (analyse_ssh.sh):

```
#!/usr/bin/env bash
set -euo pipefail
```

```
LOG_FILE="${1:-sample_auth.log}"
OUT_DIR="${2:-analysis_output}"
SCRIPT_DIR="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
AWK_FILE="$SCRIPT_DIR/analyse_ssh.awk"
if [[ ! -f "$LOG_FILE" ]]; then
    echo "Error: log file not found: $LOG_FILE" >&2
    exit 1
fi
if [[ ! -f "$AWK_FILE" ]]; then
    echo "Error: analyse_ssh.awk not found beside this script." >&2
    exit 1
fi
mkdir -p "$OUT_DIR"
# sed normalizes repeated whitespace; the original log remains unchanged.
sed -E 's/[[:space:]]+/ /g' "$LOG_FILE" > "$OUT_DIR/normalized.log"
# grep selects only SSH authentication outcomes; tee saves the filtered records.
grep -E 'sshd.*(Failed password|Invalid user|Accepted password|Accepted publickey)' \
    "$OUT_DIR/normalized.log" | tee "$OUT_DIR/ssh_events.log" >/dev/null
# awk extracts timestamps, usernames, source IPs and counts outcomes.
awk -f "$AWK_FILE" "$OUT_DIR/ssh_events.log" > "$OUT_DIR/analysis.txt"
# Power-filter pipeline: count failure records per IP using grep, cut, sort and uniq.
grep -E 'Failed password|Invalid user' "$OUT_DIR/ssh_events.log" |
    awk '{for (i=1; i<=NF; i++) if ($i=="from") print $(i+1)}' |
    sort | uniq -c | sort -nr > "$OUT_DIR/failures_by_ip.txt"
# Classify a source as suspicious at five or more failures in this sample.
awk '$1 >= 5 {print $2 " : " $1 " failed attempts (SUSPICIOUS threshold reached)"}' \
    "$OUT_DIR/failures_by_ip.txt" > "$OUT_DIR/suspicious_ips.txt"
echo "Analysis complete. Results are in: $OUT_DIR"
echo
echo "Failure counts by source IP:"
cat "$OUT_DIR/failures_by_ip.txt"
echo
echo "Sources meeting the threshold:"
if [[ -s "$OUT_DIR/suspicious_ips.txt" ]]; then
    cat "$OUT_DIR/suspicious_ips.txt"
else
    echo "None"
fi
echo
echo "Event totals:"
grep -c 'FAILED' "$OUT_DIR/analysis.txt" || true
grep -c 'SUCCESS' "$OUT_DIR/analysis.txt" || true
```

Separate AWK script (analyse_ssh.awk):

```
#!/usr/bin/awk -f
# Parse SSH authentication messages and summarize failed attempts by source IP.
```

```
{
    timestamp = $1 " " $2 " " $3
    ip = "-"
    user = "-"
    for (i = 1; i <= NF; i++) {
        if ($i == "for" && i < NF) {
            user = $(i + 1)
            if (user == "invalid" && i + 2 <= NF) user = $(i + 2)
        }
        if ($i == "from" && i < NF) ip = $(i + 1)
    }
    if ($0 ~ /Failed password|Invalid user/) {
        failures[ip]++
        total_failures++
        printf "%s,%s,%s,FAILED\n", timestamp, user, ip
    } else if ($0 ~ /Accepted password|Accepted publickey/) {
        total_success++
        printf "%s,%s,%s,SUCCESS\n", timestamp, user, ip
    }
}
END {
    print ""
    print "FAILED ATTEMPTS BY SOURCE IP"
    for (source in failures) {
        printf "%s,%d\n", source, failures[source]
    }
    printf "TOTAL_FAILED,%d\n", total_failures
    printf "TOTAL_SUCCESS,%d\n", total_success
}
```

The full files are also supplied separately in the submission folder so they can be executed directly. The report displays the key code for readability.

7. Purpose of Commands Used

#!/usr/bin/env bash – Runs the script using Bash.

set -euo pipefail – Stops on command errors, unset variables and failed pipeline components.

${1:-sample_auth.log} – Uses the first command-line argument as the log path, or a default filename.

mkdir -p – Creates the output directory if it does not already exist.

sed -E – Normalizes repeated whitespace without changing the original file.

grep -E – Filters relevant SSH authentication records using extended regular expressions.

tee – Writes filtered events to a file while passing the stream onward.

awk -f – Runs the separate AWK program against the filtered log.

sort – Orders extracted source IP addresses and numeric counts.

uniq -c – Counts adjacent repeated IP addresses after sorting.

cut -d, -f2 – Extracts the numeric total from comma-separated summary rows.

cat – Displays generated result files.

if [[-s file]] – Checks whether the suspicious-IP output file exists and is non-empty.

8. Output and Identification of Suspicious Activity

The supplied script was executed on the synthetic log. The generated failure counts are:

8 203.0.113.10
5 198.51.100.24
2 203.0.113.77

Sources meeting the threshold:

203.0.113.10 : 8 failed attempts (SUSPICIOUS threshold reached)
198.51.100.24 : 5 failed attempts (SUSPICIOUS threshold reached)

Event totals reported by the AWK analysis: 15 failed authentication events and 4 successful authentication events.

Interpretation:

- 203.0.113.10 generated 8 failed attempts, including attempts with invalid usernames such as admin, test, root, guest, oracle, postgres and support. This is consistent with repeated username guessing or account enumeration.
- 198.51.100.24 generated 5 failed attempts against the kali account, followed by a successful password authentication from the same source. This sequence deserves priority review because a successful login followed repeated failures. The synthetic log alone cannot establish whether the login was authorized.
- 203.0.113.77 generated 2 failed attempts, below the selected threshold. These attempts may still merit review if combined with other indicators.
- 192.0.2.25 generated successful public-key authentications for student. These may represent legitimate access, but authorization should be verified against the system's expected users and access schedule.

The output files are stored in analysis_output/: normalized.log, ssh_events.log, analysis.txt, failures_by_ip.txt and suspicious_ips.txt.

9. Justification of Suspicious-Event Threshold

The script classifies a source IP as suspicious when it produces five or more failed SSH authentication attempts in the supplied log sample. This is a simple demonstration threshold intended to identify repeated attempts without flagging every isolated failure. The threshold is applied to the complete sample, not to a rolling time window.

Five failures is not a universal security standard. In a real environment, the threshold should be tuned to normal user behaviour, server exposure, account lockout policy and the observation period. A

production rule could use a time window (for example, five failures within ten minutes), correlate usernames and source addresses, and raise severity when a successful login follows repeated failures.

10. False Positives and Limitations

- A legitimate user may mistype a password several times and reach the threshold.
- Automated administrative tools, monitoring systems or expired saved credentials may create repeated failures.
- A shared NAT, VPN or proxy address can represent many different users, so IP-based grouping may combine unrelated activity.
- The synthetic sample is small and intentionally constructed for demonstration; it does not represent the full complexity of real authentication logs.
- The script counts failures by source IP across the entire file. It does not implement time-window detection, geolocation, reputation checks or correlation with endpoint and firewall logs.
- Syslog formats can vary between Linux distributions and SSH versions. Changes in message wording may require updates to the grep and AWK patterns.
- A successful login after failures is a useful investigation lead, not proof of compromise. The user's authorization and surrounding activity must be checked.